
LEAN TASKS FOR WINTER VACATION

by

Jiedong Jiang & Yijun Yuan

Happy coding!
Happy Chinese New Year!

Requirements and tips. —

1. All of you work as a team to finish the tasks. When you hand in, please indicate who did what.
2. You should provide the solution both in natural language and in Lean code. Please always solve the problem in natural language first, then translate it into Lean code.
3. To train your ability to formalize the statements and search for existing results in mathlib, we will NOT provide any formalized statements in Lean, except for necessary tips.
4. Most mathematical concepts involved in the tasks can be found in mathlib. Please try to find them before defining new concepts (structure / class).
5. Any kind of tools are allowed, including but not limited to mathlib docs, Lean community on Zulip, Lean Search, Lean State Search, and LLMs. If you choose to use LLMs, we encourage you to ask question like “How to do induction in xxx situation?” instead of asking for the full solution directly.

Problem 1: Gluing continuous linear maps

Let K be a non-trivially normed field, and E, F be normed vector spaces over K . Let $\mathcal{U} = \{(E_\alpha, f_\alpha)\}_{\alpha \in I}$, where

1. I is an index set;
2. E_α is a sub-vector space of E ;
3. $f_\alpha: E_\alpha \rightarrow F$ is a continuous linear map,

satisfying the following conditions:

- (A) The union of all elements in $\mathcal{U}$ is E , i.e., $\bigcup_{\alpha \in I} E_\alpha = E$;
- (B) For any $\alpha, \beta \in I$, $f_\alpha|_{E_\alpha \cap E_\beta} = f_\beta|_{E_\alpha \cap E_\beta}$;
- (C) $(\mathcal{U}, \subseteq)$ is a directed set, i.e., for any $\alpha, \beta \in I$, there exists $\gamma \in I$ such that $E_\alpha, E_\beta \subseteq E_\gamma$.

Question 1: — Prove that there exists a unique linear map $\mathcal{F}: E \rightarrow F$ such that for any $\alpha \in I$, $\mathcal{F}|_{E_\alpha} = f_\alpha$.

Question 2: — If we further assume the following condition:

(D) There exists a constant $C > 0$ such that for any $\alpha \in I$, $\|f_\alpha\| \leq C$,

then prove that $\mathcal{F}$ is continuous.

Tips: —

1. If you try to formalize $\mathcal{U}$, you don't need to have a index set I . Try to implement $\mathcal{U}$ as a `Set` directly. You may need to figure out the type of f_α carefully.
2. You may find the following definitions in Mathlib useful: `NontriviallyNormedField`, `NormedAddCommGroup`, `NormedSpace`, `ContinuousLinearMap`, `Submodule`, `Directed`.
3. Instead of writing a `theorem` that indicates the existence of $\mathcal{F}$ directly, a better way is to define $\mathcal{F}$ by `def`, and you will be required to prove the related properties during the definition process.

Problem 2: Sequences and subsequences

Question 3: — Prove that for a non-increasing sequence of real numbers $(a_n)_{n \in \mathbb{N}}$, one and only one of the following statements holds:

1. The sequence (a_n) is eventually stable, i.e., there exists $N \in \mathbb{N}$ such that for any $n \geq N$, $a_n = a_N$;
2. There exists a subsequence of (a_n) which is strictly decreasing.

Question 4: — Let $(a_n)_{n \in \mathbb{N}}$ be a sequence of real numbers. If for any real number c , the set $\{n \in \mathbb{N} : a_n = c\}$ is finite or empty, then prove that there exists a subsequence $(a_{n_k})_{k \in \mathbb{N}}$ which makes all its terms distinct, i.e., for any $k_1 \neq k_2$, $a_{n_{k_1}} \neq a_{n_{k_2}}$.

Tips: —

1. To correctly implement the notion of subsequence, you may need to define something of type $\mathbb{N} \rightarrow \mathbb{N}$, but not $\mathbb{N} \rightarrow \mathbb{R}$. Think carefully about the reason behind this.
2. For both problems, you may need to define some thing inductively. When you try to do so in Lean, please keep the inductive structure as simple as possible, and leave the complicated parts to standalone lemmas. This will make your life much easier.
3. The `cases` tactic will be your good friend when you try to prove something that involves inductively defined objects.
4. In Problem 4, you should think about how to deliver the condition “for any real number c , the set $\{n \in \mathbb{N} : a_n = c\}$ is finite or empty” in Lean. Is `Set` really necessary here?

Problem 3: Associative composition law

Question 5: — Try to formalize the following result from Huayi's notes in Lean:

5.1.9 Proposition. *Let X be a set and $*$ be an associative composition law on X . If n is a non-zero natural number and $x_1, \dots, x_n$ are elements of X , then for any $i \in \{1, \dots, n-1\}$, the following equality holds:*

$$x_1 * \dots * x_n = x_1 * \dots * x_{i-1} * (x_i * x_{i+1}) * x_{i+2} * \dots * x_n. \quad (5.1)$$

Proof. When $i = 1$, by definition

$$x_1 * \dots * x_n = (x_1 * x_2) * \dots * x_n.$$

We suppose $i \geq 2$. By the associativity of $*$, we get

$$\begin{aligned} x_1 * \dots * x_{i+1} &= ((x_1 * \dots * x_{i-1}) * x_i) * x_{i+1} \\ &= (x_1 * \dots * x_{i-1}) * (x_i * x_{i+1}) = x_1 * \dots * x_{i-1} * (x_i * x_{i+1}). \end{aligned}$$

Hence

$$x_1 * \dots * x_n = (x_1 * \dots * x_{i+1}) * \dots * x_n = x_1 * \dots * x_{i-1} * (x_i * x_{i+1}) * \dots * x_n.$$

□

Tips: —

1. We have demonstrated to you how to define the product of a finite indexed family of elements in a type with an associative composition law. You'd better reuse that code here.
2. This might be much harder than it looks like. Try to break down the proof into several lemmas, and prove them one by one.

Problem 4: Ultrametric Normed Vector Spaces

Definition. — Let (X, d) be a metric space. We say d is **ultrametric** if for any $x, y, z \in X$, we have

$$d(x, z) \leq \max\{d(x, y), d(y, z)\}.$$

Question 6: — Let K be a non-trivially normed field, and E be an ultrametric normed vector space over K . Prove that for any sub-vector space D of F , the quotient norm on E/D is still ultrametric.

Question 7: — Let K be a non-trivially normed field, and E, F be ultrametric normed vector spaces over K . Prove that $\mathcal{L}(E, F)$, the space of continuous linear maps from E to F equipped with the operator norm, is still an ultrametric normed vector space over K .

Tips: —

1. There is already a class `IsUltrametricDist` in `mathlib`. You should use it. And the solution of the both questions should be implemented as `instances`.
2. In case you forget the definition of quotient norm or operator norm, you can find them in `mathlib` as well.

JIEDONG JIANG
YIJUN YUAN